Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Ders, dinamik bellekle matris çarpımı örneğiyle başlar. Ardından fazla parametreli fonksiyonların sorunları, eski bir alışveriş sepeti örneği, oy sayma algoritmaları ve polinomların dizide tutulması işlenir. Son bölümde struct'lara giriş yapılır.
- Dinamik matris çarpımı
  - Fonksiyonlar: `int **create_matrix(int m, int n)`, `void fill_matrix(int **matrix, int m, int n)`, `void print_matrix(...)`, `int **multiply_matrices(int **m1, int m, int n, int **m2, int k, int l)`.
  - `fill_matrix` neden `void`? Matris zaten oluşturulmuş, işaretçisi parametre olarak geliyor. Aynı işaretçiyi geri döndürmek anlamsızdır.
  - `multiply_matrices` neden işaretçi döndürüyor? Çarpım yeni bir matris üretir. Bu matris `create_matrix` ile içeride oluşturulup döndürülür, böylece kod tekrarı olmaz.
  - Boyutlar `main`'de kullanıcıdan alınır. `create_matrix` içinde alınsaydı `main` bu değerlere erişemezdi.
  - `create_matrix`: `calloc(m, sizeof(int*))`, her satır `calloc(n, sizeof(int))`. `NULL` ise `return NULL`.
  - `fill_matrix`: `matrix[i][j] = rand() % (m + n) + 1`. İşaretçi gösterimi zorunlu değil, `matrix[i][j]` yazmak daha pratik.
  - Çarpma şartı: $n = k$. Değilse `return NULL`. Sonuç $m \times l$ boyutundadır.
  - Çarpım üç iç içe döngüyle yapılır: `m3[i][j] += m1[i][x] * m2[x][j]`, $i < m$, $j < l$, $x < n$.
  - İki vektörün skaler çarpımını yapan yardımcı bir fonksiyonla (`multiply_array(a1, a2, n)`) yazmak denendi. İlk denemede sonuç yanlış çıktı: `m1[i]` bir satırdır, ama `m2`'nin sütunu bellekte bitişik değildir, bir satır gibi gönderilemez. Çözüm: `m2`'nin j. sütununu geçici bir diziye (`tmp`) kopyalayıp onu göndermek.
  - Karmaşıklık: en dış döngü m, orta l, içte kopyalama n + çarpım n = $2n$. Kare matris için $2n^3$. Sabit katsayılar ihmal edilir, yine $O(n^3)$. "n yerine $2n$ olması çok bir şey değiştirmez, n'i n^2 yapmak ayrı meseledir."
- `getopt`: komut satırı seçeneklerini (`-a`, `-b`...) ayrıştırıp `switch` ile işlemeyi sağlar. Hoca bunun "aslında Linux üzerinde yapılmış" olduğunu söyleyip Windows'ta denedi. Sorulacak bir konu olmadığını da belirtti. Örnek olarak `ls`, `dir /s` gibi komutların parametreleri gösterildi.
- İyi programlama alışkanlığı: parametre sayısı
  - Bir fonksiyona çok sayıda parametre göndermek iyi bir alışkanlık değildir. Her parametre stack'e kopyalanır.
  - Matris örneğinde `m1, m, n` üçlüsü tek bir "matris" tipiyle tek parametreye indirilebilirdi.
  - Karmaşık sayı örneği: bir fonksiyon hem gerçel hem sanal kısmı `return` ile döndüremez. Ya işaretçi parametreler kullanılır (`void get_complex(int *re, int *im)`) ya da kendi veri tipimiz tanımlanır. Polinomların katsayı ve kuvvetleri de böyledir.
- Alışveriş sepeti örneği (eski bir asistanlık örneği)
  - Fiyatlar `basket` dizisinde tutulur. Gıda ve gıda dışı ürünlerin fiyatlarının adresleri ayrı iki dizide (`ptr_food`, `ptr_other`) tutulur. Bunlar `double**`'dır: adresleri tutan diziler.
  - Başlangıçta 2 elemanlık açılır, dolunca iki katına çıkarılıp `realloc` yapılır.
  - Değerlere `*ptr_food[i]` ile erişilir. `double` olduğu için ardışık adresler 8'er artar.
- Oy sayma (seçim) algoritmaları

  - Versiyon 1: herkes 1-10 arası bir partiye oy veriyor. En çok oyu alan kim?
    * Saf yöntem: her elemanın kaç kez geçtiğini iç içe iki döngüyle saymak, $O(n^2)$.
    * Frekans dizisi: `P[oylar[i]]++`. Tek döngü, $O(n)$. Sonra `P` dizisinde en büyüğün indisi bulunur.
    * Zaman-bellek takası: 50 milyon oy ve 10 parti için saf yöntem 50 milyon × 50 milyon işlem, frekans dizisi yalnızca 10 elemanlık ek bellek ister.
  - Versiyon 2: herkes 3 partiye oy veriyor. Veri bir matris olur: `oylar[i][0..2]`. Frekans dizisi iç içe döngüyle doldurulur. Eşit oyda ilk bulunan döner; isterseniz kazananları dinamik bir dizide tutabilirsiniz.
  - Versiyon 3: herkes her partiye puan veriyor. `oylar[kişi][parti]` matrisinin sütun toplamları partilerin puanıdır.
  - Versiyon 4 fikri: oylar kişinin bir katsayısına göre ağırlıklandırılır.
- Polinomların dizide tutulması
  - İlk yöntem: dizinin indisi kuvveti, değeri katsayıyı gösterir. `{2, 4, 0, 0, -7}` → $-7x^4 + 4x + 2$.
  - Yazdırma fonksiyonu en yüksek dereceden başlar, katsayısı 0 olan terimleri atlar.
  - Değer hesaplama: `val += arr[i] * pow(x, i)`. `math.h` kullanılabilir ("regex kütüphanesi kullanmanızı beklemiyoruz ama `pow` kullanabilirsiniz"). Hoca `pow`'un kuvveti $x^{n/2}$ üzerinden n değil $\log n$ adımda hesapladığını söyledi (düzeltme için "Ders sonrası notlar"a bakın).
  - Sorun: sıfır katsayılı terimler de tutulur. İkinci yöntem: yalnızca sıfır olmayan terimler için `[katsayı, kuvvet]` çiftlerinden oluşan iki sütunlu bir matris. Ör. `{{2, 3}, {-4, 1}, {5, 4}}` → $2x^3 - 4x + 5x^4$.
  - Bu tür ilkel verilere bağımlılık (primitive obsession) kodu zorlaştırır. Çözüm: kendi veri tipimiz.
  - Proje için: kütüphane kullanımı serbest değildir ama `math.h` gibi temel fonksiyonlar kullanılabilir.
- Struct'lar
  - Kendi veri tipimizi tanımlamayı sağlar. Ör. öğrencilerin ad, soyad, yaş, vize, final bilgileri.
  - Ayrı dizilerle tutmanın sorunu: her fonksiyon tüm dizileri ayrı parametre olarak alır. Yeni bir alan (ör. doğum yeri) eklenince bütün fonksiyon tanımları ve çağrıları değişir. Struct alınca yalnızca struct değişir.
  - Nesne tabanlı programlamada (gelecek dönem) bunun gelişmiş hâli ve kalıtım (insan → öğrenci, sivil vatandaş) görülecek.
  - Tanım:
    * `struct student { int id; char name[50]; int age; char gender; };` ve `struct student s1;`
    * `typedef struct student STUDENT;` sonra `STUDENT s2;`
    * Tercih edilen: `typedef struct { ... } STUDENT;`. Kendi tiplerimizi büyük harfle yazarız.
  - Boyut ve hizalama: elemanların toplamı $4+50+4+1 = 59$ byte iken `sizeof` 64 verdi. Derleyici elemanları bellekte hizalar (alignment), araya ve sona boşluk ekler. Karakter alanlarını başa veya sona koymak, boyutları 4'ün katı seçmek (ör. 48) boşluğu azaltır. Son örnekte `char gender` başa alınıp `name[47]` yapılınca $1 + 47 + 4 + 4 = 56$ byte olur, boşluk kalmaz (gcc ile denendi).
  - Elemanlara nokta operatörüyle erişilir: `s1.id`. `scanf`'te `&s1.id`, `&s1.age`, `&s1.gender` (ampersand gerekir), `s1.name` (dizi olduğu için ampersand gerekmez).
  - Struct değişkeni fonksiyona değerle (pass by value) aktarılır: `void print_info(STUDENT s)` içine bütün içerik kopyalanır.
  - Atama: `s2 = s1;` struct'ın bütün içeriğini kopyalar. Derste `s1.id = 0` ve `strcpy(s1.name, "Deneme")` yapıldı, `s2` değişmedi.
  - Struct işaretçisi: `STUDENT *ps1 = &s1;`. `STUDENT **ps2 = &ps1;` de yazılabilir.
  - Dinamik struct dizisi: `ps1 = (STUDENT*) malloc(n * sizeof(STUDENT));`. Elemanlar `ps1[i].id` ile kullanılır, `scanf("%d", &ps1[i].id)`. İşaretçi üzerinden üyelere erişim için ayrı bir operatör vardır (gelecek ders).
  - Sabit uzunluklu struct dizisi: `STUDENT std_array[MAX];` aynı şekilde kullanılır.
  - `void print_list(STUDENT *list, int n)` hem `malloc`'la hem sabit uzunluklu dizilerle çalışır. İçinde her eleman için `print_info(list[i])` çağrılabilir. Bu bir yazılım

mühendisliği tercihidir: küçük performans kaybına karşı daha okunur ve yönetilebilir kod.
  - Bir struct kendi türünden bir üye içeremez (sonsuz, tanımı bitmemiş tip). Ama kendi türünden bir **işaretçi** içerebilir: `struct student *next;`. Bağlı listeler (linked list) bununla kurulur.

Hocanın Özellikle Vurguladığı Kısımlar

- Fonksiyon başka bir şey üretmiyorsa `void` yapın. Yeni bir yapı üretiyorsa işaretçi döndürün.
- Kod tekrarından kaçının: var olan `create_matrix` fonksiyonunu çarpım içinde de kullanın.
- Çok sayıda parametre yerine kendi veri tipinizi (struct) tanımlayın.
- Zaman-bellek takasını düşünün: frekans dizisi küçük bir bellekle büyük zaman kazandırır.
- Struct tanımını onu kullanan fonksiyonlardan önce yazın.
- `scanf` ile `%c` okumadan önce kalan Enter karakterini temizleyin.

Kısa Tekrar Notları

- Matris çarpımı: $(m \times n)(n \times l) = m \times l$, `c[i][j] += a[i][x] * b[x][j]`, $O(n^3)$.
- Frekans sayma: `P[deger]++`, $O(n)$.
- Polinom: indis = kuvvet ya da `[katsayı, kuvvet]` çiftleri.
- `typedef struct { ... } TIP;`, `s.uye`, `&s.uye` (dizi üyede & yok).
- Struct atama (`s2 = s1`) bütün içeriği kopyalar. Fonksiyona değerle aktarılır.
- Kendi türünden işaretçi üye: bağlı liste.

Detaylı Açıklamalar

Ders sonrası notlar

- **Kodların kontrolü:** Matris çarpımı (derste yazılmadı, aynı mantıkla kısa bir sürüm), struct boyutu ve atama davranışı ders sonrasında gcc ile denendi. `{int id; char name[50]; int age; char gender;}` için `sizeof` 64, `name[48]` ile 60 çıktı. Derstekiyle aynıdır.
- **Struct atamasında dizi üyesi:** Derste `s2 = s1` için "isim dizisinin yalnızca işaretçisi kopyalanıyor, ikisi aynı stringi gösteriyor" dendi. Bu doğru değildir. `char name[50]` struct'ın içinde gömülü bir dizidir. Atama bu 50 byte'ı da kopyalar; `s1` ve `s2`'nin ayrı ayrı isim dizileri olur. Derste `strcpy(s1.name, "Deneme")` sonrasında `s2.name`'in "Ahmet" olarak kalması bunu gösterir (gcc ile de aynısı doğrulandı). Yalnızca üye bir işaretçi olsaydı (`char *name`) atama işaretçiyi kopyalardı ve iki struct aynı stringi gösterirdi. Bu durumda birinde yapılan değişiklik ötekinde de görünür (sığ kopya, shallow copy).
- **Hizalama kuralı:** Derste "ikinin katına tamamlanıyor" dendi. Asıl kural, her üyenin kendi hizalama gereksinimine (int için 4 byte) göre yerleşmesi ve struct boyutunun en büyük hizalamanın katı olmasıdır. Bu yüzden boyutlar 4'ün katıdır.
- **İşaretçiyle üye erişimi:** Derste "işaretçi kullanırsak nokta değil başka bir operatör kullanılır" dendi. Bu operatör `->`'dir: `ps1->id`, `(*ps1).id` ile aynıdır. Dizi elemanına `ps1[i].id` ile nokta kullanılarak erişilir, çünkü `ps1[i]` bir struct'tır.
- **Oy sayma versiyon 3'te döngü sınırları:** Derste kullanılan kod `p[i] += oylar[j][i]` biçimindeydi. Dış döngü kişi sayısı n'ye, iç döngü parti sayısına kadar gidiyordu. `oylar[kişi][parti]` matrisinde parti toplamı için dış döngü partiler, iç döngü kişiler üzerinde olmalıdır: `for (j = 0; j < partiSay; j++) for (i = 0; i < n; i++) p[j] += oylar[i][j];`. Derste kişi ve parti sayısı ikisi de 10 olduğu için sonuç doğru çıktı; sayılar farklı olsaydı yanlış çıkardı.
- **Polinom değeri:** $2 + 4x - 7x^4$ için $P(2) = 2 + 8 - 112 = -102$'dir (gcc ile doğrulandı).
- **Sütun kopyalama sürümünde bellek:** Hoca `tmp`'yi her sütun için değil, boyutu sabit olduğu için döngülerin dışında bir kez `malloc` ile açtı (00:38:06-00:38:28). `free(tmp)` yazılmazsa fonksiyonun her çağrısında n tam sayılık alan sızar, bu yüzden fonksiyonun sonunda `free(tmp)` gerekir. En basit ve önerilen yol üç iç içe döngüdür.

- **`pow` ve** $\log n$**:** Derste anlatılan $x^{n/2}$ yöntemi hızlı üs almadır ve tam sayı üslere uyar. Standart kütüphanedeki `pow double` üs aldığı için büyük olasılıkla farklı bir yöntemle (logaritma ve üstel yaklaşım) yazılmıştır. Gerçekleme kütüphaneye bağlıdır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.